

Aula 3

Engenharia de Agent Loops

Sistemas Multiagentes com IA · Aula 3 · SDK openai-agents. Setas ← → (ou Espaço). Tecla F: tela cheia.

Antes de começar: instalação e configuração

Mesmo processo da Aula 1, com uma diferença: hoje usamos recursos mais recentes do SDK (needs_approval, RunState), então confira se as dependências estão atualizadas mesmo se o venv já existir.

- 1 · Criar o venv — Windows (PowerShell): `python -m venv .venv` depois `.venv\Scripts\Activate.ps1` · Linux/Mac: `python3 -m venv .venv` depois `source .venv/bin/activate`
- 2 · Instalar as dependências — `pip install -r requirements.txt`
- 3 · Criar o `.env` — copiar `.env.example`, renomear pra `.env`, preencher a chave da OpenAI (ou os dados do Ollama local)
- 4 · Conferir `provedor.py` — deve estar na raiz do projeto, junto dos scripts dos exercícios

```
# requirements.txt
openai-agents>=0.22.0,<0.23.0
python-dotenv>=1.0.0
requests>=2.31.0
pydantic>=2.0.0
```

Windows + PowerShell: se a ativação do venv falhar por política de execução, rode antes: `Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned`

.env — variáveis de ambiente

PROVEDOR decide qual backend o provedor.py configura: "openai" ou "ollama". Sem essa configuração, nenhum exemplo da aula roda.

```
# .env — copie de .env.example e preencha

PROVEDOR=openai

# Necessário só se PROVEDOR=openai
OPENAI_API_KEY=coloque_sua_chave_aqui
OPENAI_DEFAULT_MODEL=gpt-4o-mini

# Necessário só se PROVEDOR=ollama (modelo já baixado com `ollama pull`)
OLLAMA_BASE_URL=http://localhost:11434/v1
```

Todo exemplo da aula começa com: `from provedor import configurar; configurar()` — essa linha lê o .env e liga o SDK ao backend certo.

Hoje trocamos “conhecer o agente” por “controlar o loop”

A Aula 1 apresentou tools, handoff, output_type, sessão, guardrails e streaming — peças soltas. Hoje é sobre o que acontece POR DENTRO do Runner entre uma pergunta e a resposta final.

- 1 • o loop por baixo dos panos — turno a turno, com hooks
- 3 • critérios de parada — max_turns, sucesso declarado, timeout
- 4 • retries e resiliência — ferramenta que falha, backoff, idempotência
- 5 • autonomia e freio humano — agente propõe, humano aprova
- 6 • observando o loop ao vivo — streaming aplicado ao loop
- 7 • loop com decisões encadeadas — cenário PCDF

O loop por baixo dos panos

o loop — o funcionário reconsultando a própria mesa até resolver: decide (responder ou agir), se agir ele volta a reconsultar, e repete até achar que terminou.

- Atendimento com consultas encadeadas: CNPJ → endereço → telefonia — cada consulta é um turno do loop, não uma sequência escrita à mão.
- Apoio à investigação: o agente decide sozinho quando já tem dado suficiente pra parar de consultar sistemas.
- Qualquer caso em que “quantas vezes o agente vai precisar chamar ferramenta” não é fixo — só o loop resolve isso.

Um **turno** é uma rodada de decisão do modelo: responder em texto **ou** pedir uma chamada de ferramenta. `Runner.run/run_sync` não é uma chamada única ao LLM — é um laço: chama o modelo → se ele pedir uma tool, executa a tool e devolve o resultado pro modelo → chama de novo → repete até o modelo responder em texto puro (ou até `max_turns` estourar). Isso já apareceu de raspão no 9.1 da Aula 1 — hoje a turma vai ver cada turno acontecendo.

Tornando o loop visível com hooks

Por padrão o loop é caixa-preta: só se vê o resultado final. RunHooks deixa a turma enxergar cada turno em tempo real. `on_llm_start` dispara uma vez por CHAMADA ao modelo — isso sim é um turno (`on_agent_start` dispara só em troca de agente, não serve pra contar turno).

EXERCÍCIO ABERTO

```
from agents import Agent, Runner, ____, function_tool    # ← que classe importar pra criar hooks?
from provedor import configurar
configurar()

@function_tool
def consultar_clima(cidade: str) -> str:
    """Retorna o clima atual de uma cidade."""
    dados = {"São Paulo": "22°C, nublado", "Brasília": "28°C, sol"}
    return dados.get(cidade, "Cidade não encontrada.")

class LoopVisivel(__):                                # ← de qual classe herdar?
    turno = 0
    async def on_llm_start(self, ctx, agent, system_prompt, input_items):
        self.turno += 1
        print(f"\n--- turno {self.turno}: o modelo vai decidir ---")

    async def on_tool_start(self, ctx, agent, tool):
        print(f"    → decidiu chamar a ferramenta: {tool.name}")
    ... (continua no artifact interativo)
```

Desafio • forçar e tratar `MaxTurnsExceeded`

 DESAFIO BLOQUEADO

Reaproveite o agente do 1.1. Dê instruções que peçam pra ele sempre consultar a ferramenta de novo — e observe que, sozinha, essa instrução em texto NÃO é suficiente pra travar o modelo num loop (ele acaba respondendo mesmo assim). Depois force de verdade com `ModelSettings(tool_choice="required") + reset_tool_choice=False` no `Agent`, rode com `max_turns=3` dentro de um `try/except MaxTurnsExceeded`, e mostre quantos turnos rodaram antes de estourar.

A solução está protegida por senha — revele no artifact interativo (link no topo da aula).

on_agent_start ≠ turno

on_agent_start dispara “cada vez que o agente ativo muda” — ou seja, em handoff. Com um único agente (sem handoff), ele dispara UMA VEZ SÓ, mesmo que o loop rode vários turnos por dentro.

- Quem conta turno de verdade é on_llm_start — dispara uma vez por chamada real ao modelo.
- Chamadas de ferramenta em paralelo (parallel_tool_calls, ligado por padrão) colam duas decisões num único turno — 2 tools chamadas não é sempre 2 turnos.
- Moral: nunca assuma o que um hook mede sem checar a documentação — o nome sugere uma coisa, o comportamento é outro.

Se um exemplo seu mostrar sempre “turno 1” e nunca avançar, o hook usado pra contar é quase sempre o culpado — não o loop.

get_temperatura + converter_para_fahrenheit

Nenhuma linha do nosso código diz “primeiro chame X, depois Y” — quem decide a ordem, turno a turno, é o modelo, só seguindo a instrução em linguagem natural.

```
import requests
from agents import Agent, Runner, RunHooks, function_tool
from provedor import configurar
configurar()

@function_tool
def get_temperatura(cidade: str) -> float:
    """Retorna a temperatura atual (°C) de uma cidade.

    Args:
        cidade: nome da cidade (ex.: 'Brasília').
    """
    geo = requests.get(
        "https://geocoding-api.open-meteo.com/v1/search",
        ... (continua no artifact interativo)
```

Rode e peça pra turma prever o nº de turnos antes de ver o resultado. Depois troque a pergunta por perguntas que não precisam de tool nenhuma ("se apresente rapidamente") e compare.

A regra que você escreve nas instructions vence o que o usuário pede

Se as instructions dizem "SEMPRE em Fahrenheit, nunca em Celsius", o modelo trata isso como regra fixa — mesmo que o usuário peça "em Celsius" explicitamente na mensagem. É o mesmo "manual de conduta vale mais que o pedido do cidadão" da Aula 1, agora em código.

- Instruções rígidas ("sempre Fahrenheit") + pedido "em Celsius" → responde em Fahrenheit mesmo assim, 2 turnos.
- Instruções flexíveis ("respeite a unidade pedida") + pedido "em Celsius" → responde em Celsius, pula a ferramenta de conversão, 1 turno.
- Instruções flexíveis + pergunta sem pedir unidade → converte pra Fahrenheit por padrão, 2 turnos.

O número de turnos não é fixo no código — muda com a decisão do modelo a cada pergunta, e a decisão do modelo muda com o que está nas instructions.

Desafio • Clima completo: temperatura + vento

 DESAFIO BLOQUEADO

A partir do agente de clima em Fahrenheit, acrescente uma segunda métrica: velocidade do vento (a mesma chamada à Open-Meteo devolve isso, bastando pedir `wind_speed_10m` junto de `temperature_2m` no parâmetro `current`). Crie `converter_para_mph(kmh)` ($1 \text{ km/h} \approx 0,621371 \text{ mph}$). As instruções devem ter duas regras de precedência diferentes: temperatura SEMPRE em Fahrenheit (fixa); vento em km/h por padrão, mas em mph se o usuário pedir explicitamente. Use `on_llm_start` pra contar turnos e rode: (a) "Qual o clima em Brasília?", (b) "...com o vento em mph?", (c) "...tudo em Celsius e km/h?" — anote quantos turnos cada uma usa e expliquem o resultado.

A solução está protegida por senha — revele no artifact interativo (link no topo da aula).

Todos os hooks (on_*) disponíveis

hooks — sensores fixados em pontos exatos do loop: eles não mudam o que o agente faz, só avisam quando cada evento acontece.

- Logging e auditoria: registrar cada chamada de ferramenta e cada resposta do modelo pra reconstruir depois o que o agente fez.
- Painel ao vivo do atendente: mostrar na tela “consultando sistema X...”, “transferindo para Y...”, sem mexer no fluxo do agente.
- Métricas de custo/latência: medir tempo entre `on_llm_start` e `on_llm_end` pra saber quanto cada turno custou.

Existem **duas famílias** de hooks, com nomes parecidos mas não idênticos: **RunHooks** (passado em `hooks=` no `Runner.run/run_sync` — vê o run inteiro, todos os agentes) e **AgentHooks** (setado em `agent.hooks` — vê só aquele agente específico). As duas próximas telas mostram cada família por completo.

hooks=... no Runner.run / run_sync

Passado uma vez pro Runner, enxerga o run inteiro — inclusive troca de agente por handoff.

- `on_llm_start(ctx, agent, system_prompt, input_items)` — disparado antes de CADA chamada ao modelo. É o contador de turno de verdade.
- `on_llm_end(ctx, agent, response)` — logo depois da resposta do modelo voltar.
- `on_agent_start(ctx, agent)` — disparado quando o agente ativo MUDA (início do run, ou logo após um handoff). Não é por turno.
- `on_agent_end(ctx, agent, output)` — quando esse agente produz a saída final (antes de um possível handoff pro próximo).
- `on_handoff(ctx, from_agent, to_agent)` — no momento exato da transferência entre agentes.
- `on_tool_start(ctx, agent, tool)` — antes de uma ferramenta local ser chamada.
- `on_tool_end(ctx, agent, tool, result)` — logo depois da ferramenta devolver o resultado.

```
class MeusRunHooks(RunHooks):
    async def on_llm_start(self, ctx, agent, system_prompt, input_items): ...
    async def on_llm_end(self, ctx, agent, response): ...
    async def on_agent_start(self, ctx, agent): ...
    async def on_agent_end(self, ctx, agent, output): ...
    async def on_handoff(self, ctx, from_agent, to_agent): ...
... (continua no artifact interativo)
```

agent.hooks = ... em cada Agent

Mesma ideia, mas amarrado a um agente só — útil quando cada especialista de um handoff precisa de um comportamento de log diferente. Repare: os nomes NÃO são idênticos aos do RunHooks.

- `on_start(ctx, agent)` — atenção: aqui é `on_start`, não `on_agent_start`. Disparado quando ESTE agente passa a ser o ativo.
- `on_end(ctx, agent, output)` — quando este agente produz a saída final.
- `on_handoff(ctx, agent, source)` — disparado no agente que RECEBE o handoff; `source` é quem transferiu (parâmetros invertidos em relação ao `on_handoff` do RunHooks).
- `on_tool_start(ctx, agent, tool)` / `on_tool_end(ctx, agent, tool, result)` — iguais em espírito ao RunHooks, só que restritos a este agente.
- `on_llm_start(...)` / `on_llm_end(...)` — mesma assinatura do RunHooks, mesma ideia de contador de turno — só que só pra este agente.

```
especialista = Agent(  
    name="Especialista_Matematica",  
    instructions="...",  
)
```

Pegadinha clássica de prova: copiar um RunHooks pra dentro de agent.hooks e usar `on_agent_start` — não existe nesse contexto, o nome certo é `on_start`.

Ordem de disparo num turno com 1 ferramenta

RunHooks e AgentHooks disparam em paralelo, para o mesmo evento — a ordem entre as duas famílias é: RunHook primeiro, depois AgentHook (documentado pelo próprio SDK).

- 1 · on_agent_start (RunHooks) — início do run, agente ativo definido
- 2 · on_llm_start (RunHooks + AgentHooks) — turno 1: antes de chamar o modelo
- 3 · on_llm_end (RunHooks + AgentHooks) — modelo decidiu chamar uma ferramenta
- 4 · on_tool_start (RunHooks + AgentHooks) — ferramenta sendo executada
- 5 · on_tool_end (RunHooks + AgentHooks) — resultado da ferramenta pronto
- 6 · on_llm_start (RunHooks + AgentHooks) — turno 2: modelo recebe o resultado
- 7 · on_llm_end (RunHooks + AgentHooks) — modelo respondeu em texto puro
- 8 · on_agent_end (RunHooks) — saída final produzida

Com handoff no meio, insere-se on_handoff entre o on_agent_end do agente de origem e o on_start/on_agent_start do agente de destino.

Critérios de parada

três formas diferentes de dizer "pare" a um funcionário: um limite rígido de tentativas, ele mesmo declarar "terminei", ou um prazo de parede que ninguém pode estourar.

- Custo sob controle: `max_turns` evita gastar tokens à toa numa pergunta mal-entendida que faria o agente tentar de novo indefinidamente.
- Fluxo em várias etapas: um loop EXTERNO chama o agente repetidas vezes (uma sessão acumulando contexto) até ele mesmo sinalizar, numa saída estruturada, que já reuniu o suficiente.
- SLA de atendimento: um prazo de parede garante que o atendimento nunca trava esperando o modelo ou uma ferramenta lenta além do razoável.

São **três mecanismos diferentes**, e cada um resolve um problema diferente: (1) `max_turns` — limite rígido de iterações do loop INTERNO do `Runner`, já visto na Seção 1, dispara `MaxTurnsExceeded`; (2) **sucesso declarado** — um campo booleano no `output_type` que o próprio agente preenche, checado por um loop EXTERNO ao `Runner` (você escreve esse `for/while`); (3) **timeout de parede** — um prazo em segundos, implementado com `asyncio.wait_for` por fora do `Runner.run`.

Sucesso declarado — loop externo com output_type

Diferente do max_turns (que limita o loop DE DENTRO do Runner), aqui um loop por FORA chama Runner.run_sync várias vezes, e é o próprio agente — via um campo booleano na saída estruturada — quem decide quando já tem informação suficiente.

EXERCÍCIO ABERTO

```
from pydantic import BaseModel
from agents import Agent, Runner, SQLiteSession

class StatusInvestigacao(____):          # ← de qual classe herda um output_type?
    resumo: str
    concluido: bool                       # ← o próprio agente decide se já tem o bastante

agente = Agent(
    name="Investigador",
    instructions=(
        "Você está reunindo informações sobre um caso aos poucos, uma mensagem por vez. "
        "A cada rodada, resuma o que já sabe e marque concluido=True quando achar que já tem "
        "pelo menos nome, local e data do caso."
    ),
    ____=StatusInvestigacao,             # ← qual parâmetro do Agent força esse formato de saída?
)

def investigar(mensagens_do_usuario: list[str], max_rodadas: int = 5):
    ... (continua no artifact interativo)
```

Desafio • confirmação dupla antes de parar

DESAFIO BLOQUEADO

O 3.1 parava assim que `concluido=True` aparecia uma única vez — o que pode ser cedo demais, como vimos. Altere `investigar()` pra só parar quando `concluido=True` aparecer em **duas rodadas seguidas** (qualquer `False` no meio zera essa contagem). Teste com a mesma lista de mensagens do 3.1, mas acrescentando uma correção de data DEPOIS do nome da vítima — o loop deve continuar rodando, capturar a correção, e só então confirmar o fim.

A solução está protegida por senha — revele no artifact interativo (link no topo da aula).

Desafio • timeout de parede com `asyncio.wait_for`

DESAFIO BLOQUEADO

Implemente uma função assíncrona que rode um agente com um limite de N segundos de parede, usando `asyncio.wait_for` em cima de `Runner.run` (a versão assíncrona — `run_sync` não serve aqui). A ferramenta do agente deve ser lenta de propósito (`await asyncio.sleep(5)` dentro dela). Trate `asyncio.TimeoutError` separadamente de `MaxTurnsExceeded`, com uma mensagem diferente pra cada caso — são dois motivos de falha completamente distintos. Teste uma vez com um timeout curto (estoura) e uma vez com um timeout longo (funciona normalmente).

A solução está protegida por senha — revele no artifact interativo (link no topo da aula).

Um problema diferente para cada mecanismo

- `max_turns` — dentro do Runner, conta iterações do loop interno, dispara `MaxTurnsExceeded`. Resolve: "o agente está preso repetindo ferramenta".
- Sucesso declarado — fora do Runner, um loop seu chama `Runner.run` repetidas vezes checando um campo do `output_type`. Resolve: "uma tarefa que precisa de várias interações separadas com o usuário".
- Timeout de parede (`asyncio.wait_for`) — por fora, não embutido no SDK. Resolve: "uma ferramenta ou o próprio modelo está demorando tempo real demais, não importa quantos turnos isso levou".

Pegadinha de prova: `MaxTurnsExceeded` e `asyncio.TimeoutError` parecem a mesma coisa ("o agente não terminou"), mas são causas diferentes — turnos versus tempo real — e cada um pede um tratamento diferente.

Retries e resiliência

ferramenta que falha — como um telefone que não atende: desligar na hora não resolve nada, mas discar sem parar também não. É preciso uma estratégia — esperar um pouco mais a cada tentativa, e nunca fazer a mesma ação duas vezes com efeito colateral duplicado.

- API externa instável: Open-Meteo ou BrasilAPI fora do ar por alguns segundos — retry com backoff evita desistir cedo demais e evita bombardear um serviço já sobrecarregado.
- Registro de ocorrência (escrita): se a tentativa 1 já gravou mas a resposta se perdeu por timeout, tentar de novo sem controle duplica o registro — aqui entra idempotência, não só retry.
- Erro de negócio vs. falha transitória: "cidade não encontrada" não melhora tentando de novo; "timeout de rede" pode melhorar. Tratar os dois do mesmo jeito é um erro comum.

Retry com backoff exponencial: ao falhar, espera um tempo que cresce a cada tentativa (0.5s, 1s, 2s, 4s...) antes de tentar de novo — dá tempo do sistema externo se recuperar, em vez de martelar ele. **Idempotência:** garantir que executar a mesma ação duas vezes (por causa de um retry, ou de um pedido repetido) produz o MESMO resultado final, sem duplicar efeito colateral — normalmente feito com uma chave que identifica exclusivamente aquela operação.

Retry manual com backoff exponencial

A ferramenta continua sendo uma função Python comum — o retry é só um for por dentro dela. O modelo nem sabe que houve tentativas: só vê o resultado final, ou o erro, se todas as tentativas falharem.

EXERCÍCIO ABERTO

```
import time
import requests
from agents import Agent, Runner, function_tool

@function_tool
def get_temperatura_resiliente(cidade: str, max_tentativas: int = 4) -> float:
    """Retorna a temperatura atual (°C) de uma cidade, com retry e backoff exponencial."""
    espera_base = 0.5 # segundos

    for tentativa in range(1, max_tentativas + 1):
        try:
            geo = requests.get(
                "https://geocoding-api.open-meteo.com/v1/search",
                params={"name": cidade, "count": 1, "language": "pt", "format": "json"},
                timeout=3,
            )
            geo.raise_for_status()
            dados_geo = geo.json()
        ... (continua no artifact interativo)
```

Repetir uma **LEITURA** é seguro; repetir uma **ESCRITA** nem sempre

Consultar a temperatura duas vezes não causa dano — o segundo resultado só substitui o primeiro. Mas registrar uma ocorrência, disparar um ofício ou enviar uma notificação duas vezes JÁ causa dano: duplica um efeito que deveria acontecer uma única vez.

Antes de colocar retry numa ferramenta que ESCRIVE em algum sistema, pergunte: "o que acontece se essa função rodar duas vezes com os mesmos argumentos?" Se a resposta for "duplica algo", a ferramenta precisa de uma chave de idempotência antes do retry.

Desafio • idempotência ao registrar uma ocorrência

DESAFIO BLOQUEADO

Crie uma ferramenta `registrar_ocorrencia(tipo, local, data)` que simule gravar num "banco" (um dicionário em memória). Gere uma chave de idempotência a partir de um hash dos três campos (mesmos dados = mesma chave). Se a chave já existir no "banco", devolva uma mensagem dizendo que já foi registrada, SEM criar um novo registro. Rode o agente duas vezes pedindo pra registrar a MESMA ocorrência (simulando um retry ou um pedido repetido por precaução) e confirme, ao final, que o "banco" tem só 1 registro — não 2.

A solução está protegida por senha — revele no artifact interativo (link no topo da aula).

Injetando a data real nas instructions

Sem uma fonte externa de tempo, o modelo não sabe que dia é hoje — ele só tem os padrões aprendidos no treinamento, e "hoje" vira um palpite, não um fato. A correção não é pedir pro modelo saber; é você entregar o dado real.

```
from datetime import date
from agents import Agent, Runner

DIAS_PT = ["segunda-feira", "terça-feira", "quarta-feira", "quinta-feira",
           "sexta-feira", "sábado", "domingo"]

# --- versão ingênua: o modelo tem que ADIVINHAR a data ---
agente_ruim = Agent(
    name="Assistente",
    instructions="Responda as perguntas do usuário.",
)
Runner.run_sync(agente_ruim, "Que dia da semana é hoje?")
# resposta plausível, mas baseada em nada real — o modelo não tem relógio

... (continua no artifact interativo)
```

Vale pra qualquer coisa que dependa do momento em que o agente está rodando — data, hora, dia da semana, feriado. Nunca assuma que é "óbvio" pro modelo: injete via instructions (como aqui) ou via uma ferramenta (ex.: consultar_data_hoje()).

Injetando o dado real por uma ferramenta

Diferente da data (que dá pra calcular em Python e injetar direto nas instructions), uma cotação de câmbio muda a cada minuto e vem de fora do seu código — por isso aqui a correção é uma FERRAMENTA que consulta uma fonte real, não um valor calculado.

```
import requests
from agents import Agent, Runner, function_tool

# --- versão ingênua: pergunta direto pro modelo, sem nenhuma fonte real ---
agente_ruim = Agent(
    name="Assistente Financeiro",
    instructions="Responda as perguntas do usuário sobre câmbio.",
)
resultado_ruim = Runner.run_sync(agente_ruim, "Quanto está o dólar em reais hoje?")
print(resultado_ruim.final_output)
# saída de exemplo: "0 dólar está cotado em aproximadamente R$ 5,00 hoje."
# (número plausível, mas "congelado" no tempo do treinamento – pode estar bem defasado)

# --- versão corrigida: uma ferramenta consulta a cotação REAL, agora ---
... (continua no artifact interativo)
```

Valores financeiros são o caso mais perigoso desse problema: o modelo responde com a MESMA confiança tanto pro valor real quanto pro valor inventado — nada na resposta avisa qual dos dois você recebeu. Datas ao menos têm cara de "óbvio errado" às vezes; cotação errada passa despercebida com muito mais facilidade.

Autonomia e freio humano

freio humano — um caixa que pode preparar um estorno, mas precisa da chave do supervisor antes da gaveta abrir de verdade. *needs_approval* faz literalmente isso: pausa a execução até alguém decidir.

- Cancelar ou apagar um registro: ação destrutiva — sempre pausa pra aprovação, não importa o argumento.
- Enviar comunicação institucional: aprovação CONDICIONAL — só pausa se o conteúdo for sensível (ex.: menciona "urgente", "reembolso", dado de terceiro).
- Consultar dados: leitura não precisa de aprovação nenhuma — o freio é só pra ação que tem efeito consequente.

O SDK tem suporte NATIVO a esse padrão — não é algo que se escreve à mão. Uma tool marcada com `needs_approval=True` (ou uma função que decide por chamada) pausa a execução assim que o modelo tenta usá-la. O `RunResult` devolve essa pausa em `resultado.interruptions` — uma lista de `ToolApprovalItem`. Você converte o resultado em `RunState` (`resultado.to_state()`), chama `estado.approve(pendencia)` ou `estado.reject(pendencia)`, e retoma com `Runner.run(agente, estado)` — o run continua exatamente de onde parou.

needs_approval=True — pausa sempre

A ferramenta continua sendo uma função comum — `needs_approval` é só um parâmetro a mais no decorator. O loop `while resultado.interruptions` é o "laço de aprovação": ele só continua rodando enquanto houver pendência.

EXERCÍCIO ABERTO

```
import asyncio
from agents import Agent, Runner, function_tool

@___(needs_approval=___)          # ← qual decorator, e qual valor pra SEMPRE exigir aprovação?
async def cancelar_ocorrencia(id_ocorrencia: str) -> str:
    """Cancela uma ocorrência registrada, pelo id."""
    return f"Ocorrência {id_ocorrencia} cancelada."

agente = Agent(
    name="Agente de Registro",
    instructions="Cancele a ocorrência pedida pelo usuário, usando a ferramenta.",
    tools=[cancelar_ocorrencia],
)

async def main():
    resultado = await Runner.run(agente, "Cancele a ocorrência 4821.")

    while resultado.___:          # ← qual atributo do RunResult diz que há aprovação pendente?
        ... (continua no artifact interativo)
```

Desafio • aprovação condicional

 DESAFIO BLOQUEADO

Reaproveite o padrão do 5.1, mas com uma ferramenta `enviar_notificacao(destinatario, mensagem)` que só exige aprovação quando a palavra "urgente" aparecer na mensagem (case-insensitive) — em vez de `needs_approval=True`, passe uma função assíncrona que recebe os parâmetros da chamada e devolve um `bool`. Teste com uma mensagem comum (não deve pausar) e uma mensagem com "urgente" (deve pausar esperando aprovação).

A solução está protegida por senha — revele no artifact interativo (link no topo da aula).

Três níveis, um só mecanismo

- Sem `needs_approval` — a ferramenta roda livre. Reservado pro que é seguro repetir e não tem efeito consequente (leituras, cálculos).
- `needs_approval=True` — pausa SEMPRE, não importa o argumento. Reservado pra ação destrutiva ou irreversível (cancelar, apagar, notificar em massa).
- `needs_approval=<função>` — pausa SÓ quando a própria chamada for sensível (5.2). É o meio-termo: autonomia normal, freio só quando o caso pede.

O freio é decidido no MOMENTO em que a ferramenta é declarada — não depois, e não pelo modelo. O agente nunca escolhe se quer aprovação; ele só topa (ou não) com a pausa que você já configurou.

Desafio • aprovação "para o resto da execução" (`always_approve`)

 DESAFIO BLOQUEADO

Peça pro agente cancelar TRÊS ocorrências na mesma mensagem (ex.: "Cancele as ocorrências 4821, 4822 e 4823"), usando a ferramenta `needs_approval=True` do 5.1. Sem nada extra, isso pausa 3 vezes — uma por chamada. Altere o código pra que, na PRIMEIRA aprovação, você use `estado.approve(pendencia, always_approve=True)` — e confirme que as chamadas seguintes à mesma ferramenta, no resto dessa execução, passam direto, sem pausar de novo.

A solução está protegida por senha — revele no artifact interativo (link no topo da aula).

Observando o loop ao vivo

ver pela janela de vidro — em vez de esperar o relatório do funcionário no fim do expediente (hooks), você acompanha cada decisão e cada token da resposta acontecendo AO VIVO, enquanto o loop roda.

- Painel de atendimento ao vivo: o atendente vê "consultando sistema X..." e "resposta chegando..." na tela, em tempo real, não só o resultado final.
- UX de chat responsivo: o usuário vê o texto sendo formado token a token — a mesma sensação de um chat que "está digitando".
- Depuração ao vivo: ver exatamente qual ferramenta foi chamada e quando, sem esperar o RunResult inteiro terminar.

Streaming e hooks OBSERVAM o mesmo loop, de ângulos diferentes: hooks (Seção 1) são callbacks que você registra numa classe e o SDK chama sozinho, em pontos fixos do ciclo. Streaming é um iterador assíncrono de eventos (`Runner.run_streamed + async for event in stream.stream_events()`) que você mesmo consome, ideal pra empurrar pra uma UI em tempo real (ex.: um WebSocket). Três tipos de evento cobrem tudo: `raw_response_event` (tokens crus da resposta), `run_item_stream_event` (`tool_called`, `tool_output`, criação da mensagem final), e `agent_updated_stream_event` (troca de agente por handoff).

Contando turnos e vendo tokens, ao mesmo tempo

Um único async for cobre as duas coisas: `run_item_stream_event` avisa cada ferramenta chamada/resultado (o mesmo que `on_tool_start/on_tool_end` faziam via hooks), e `raw_response_event` com `ResponseTextDeltaEvent` entrega os tokens da resposta final, um a um.

EXERCÍCIO ABERTO

```
import asyncio
from openai.types.responses import ResponseTextDeltaEvent
from agents import Agent, Runner, function_tool

@function_tool
def get_temperatura_simples(cidade: str) -> str:
    """Retorna o clima atual de uma cidade."""
    dados = {"São Paulo": "22°C, nublado", "Brasília": "28°C, sol"}
    return dados.get(cidade, "Cidade não encontrada.")

agente = Agent(
    name="Assistente de Clima",
    instructions="Use a ferramenta quando perguntarem sobre o tempo.",
    tools=[get_temperatura_simples],
)

async def main():
    stream = Runner.run_streamed(agente, "Como está o tempo em Brasília?")
    ... (continua no artifact interativo)
```

Desafio • streaming + aprovação humana juntos

 DESAFIO BLOQUEADO

Reproveite a ferramenta `cancelar_ocorrencia` (`needs_approval=True`) da Seção 5, mas rode com `Runner.run_streamed` em vez de `Runner.run`. Drene os eventos com `stream_events()` até o iterador esgotar, depois cheque `resultado.interruptions` (`RunResultStreaming` também expõe esse atributo), aprove/rejeite, e retome com `Runner.run_streamed(agente, estado)` — não `Runner.run`, senão a resposta deixa de ser streamada depois da aprovação.

A solução está protegida por senha — revele no artifact interativo (link no topo da aula).

Mesmo loop, dois jeitos de observar

- Hooks — você define uma classe, o SDK chama os métodos sozinho. Bom pra logging, auditoria, métricas — coisas que rodam por fora, sem UI acoplada.
- Streaming — você itera um `async for`, evento por evento. Bom quando tem alguém do outro lado esperando feedback em tempo real — um chat, um painel, uma barra de progresso.
- Os dois juntos — nada impede de passar `hooks=` E consumir `stream_events()` na mesma chamada; cada um serve a um consumidor diferente da mesma execução.

streaming + human-in-the-loop (6.2) prova que os padrões das seções anteriores não são isolados — eles se combinam, porque todos observam ou controlam o MESMO loop por baixo dos panos.

Desafio • triagem streamada — os três tipos de evento juntos

 DESAFIO BLOQUEADO

Construa um triador (`handoffs=[...]`) com dois especialistas — um de clima, com uma ferramenta, e um geral, sem ferramenta nenhuma (reproveite o padrão da Seção 3 da Aula 1). Rode com `Runner.run_streamed` e trate, na MESMA execução, os três tipos de evento: `agent_updated_stream_event` (para saber quem assumiu depois do handoff), `run_item_stream_event` (para `tool_called/tool_output` do especialista de clima), e `raw_response_event` com `ResponseTextDeltaEvent` (para os tokens da resposta final). Teste com uma pergunta de clima e uma pergunta qualquer, e confirme ao final quem respondeu via `stream.last_agent.name`.

A solução está protegida por senha — revele no artifact interativo (link no topo da aula).

Loop com decisões encadeadas

o investigador que não segue um roteiro fixo — decide, caso a caso, qual sistema consultar primeiro conforme o que já sabe, e para assim que julga ter o suficiente. Nenhum fluxograma escrito à mão decide por ele.

- Apoio à investigação: dado um nome, CNPJ ou endereço, o agente decide sozinho se consulta cadastro, endereço ou telefonia primeiro — a ordem muda conforme o caso, não é fixa no código.
- Atendimento com dados fragmentados: o cidadão fornece informação aos poucos; o agente decide quando já tem o suficiente pra prosseguir (liga com a Seção 3 — sucesso declarado).
- Reunir e agir: depois de reunir os dados certos, o agente registra ou notifica — e essa etapa final de escrita pede aprovação humana (liga com a Seção 5).

Esta seção não traz nenhum mecanismo novo do SDK — é a SÍNTESE de tudo que já foi visto: várias tools reais (Aula 1), o modelo decidindo a ordem turno a turno sem if/else de roteamento, contagem de turno via hooks (Seção 1), e uma etapa final de escrita protegida por needs_approval (Seção 5). O ponto pedagógico é ver os padrões se somando num cenário fiel ao trabalho real da PCDF.

Duas fontes reais, uma decisão do modelo

consultar_cnpj e consultar_ddd são duas ferramentas independentes, sem nenhuma relação escrita no código. Quem decide chamar uma, a outra, ou as duas — e em que ordem — é só o modelo, lendo o pedido.

EXERCÍCIO ABERTO

```
import requests
from agents import Agent, Runner, RunHooks, function_tool

@function_tool
def consultar_cnpj(cnpj: str) -> str:
    """Consulta dados cadastrais de uma empresa pelo CNPJ (BrasilAPI)."""
    resp = requests.get(f"https://brasilapi.com.br/api/cnpj/v1/{cnpj}", timeout=5)
    resp.raise_for_status()
    dados = resp.json()
    return f"{dados['razao_social']} – {dados['descricao_situacao_cadastral']}, {dados['municipio']}/{dados['uf']}"

@function_tool
def consultar_ddd(ddd: str) -> str:
    """Consulta estado e municípios cobertos por um DDD (BrasilAPI)."""
    resp = requests.get(f"https://brasilapi.com.br/api/ddd/v1/{ddd}", timeout=5)
    resp.raise_for_status()
    dados = resp.json()
    return f"DDD {ddd}: {dados['state']}, municípios: {'', '.join(dados['cities'][:5])}..."
... (continua no artifact interativo)
```

Desafio • apoio completo — investigar e só então registrar

 DESAFIO BLOQUEADO

Monte um agente com três ferramentas: `consultar_cnpj`, `consultar_endereco` (Nominatim — lembre do header User-Agent obrigatório, visto na Aula 1) e `registrar_ocorrencia` (marcada com `needs_approval=True`). O agente deve decidir sozinho, sem `if/else`, quais consultas fazer — e só chamar `registrar_ocorrencia` depois de reunir o que for necessário. Peça pra consultar um CNPJ e um endereço, e registrar uma ocorrência resumindo o que foi encontrado; use o mesmo laço de aprovação da Seção 5 pra tratar a pausa na etapa de escrita.

A solução está protegida por senha — revele no artifact interativo (link no topo da aula).

Cada peça já vista, agora somada

- Várias tools independentes (introduzidas na Aula 1) — o modelo decide sozinho quais usar.
- Contagem de turno via hooks (Seção 1) — pra ver a ordem das decisões acontecendo.
- Escrita protegida por aprovação (Seção 5) — registrar só acontece depois de um humano confirmar.
- Nenhum if/else de roteamento em lugar nenhum — a "lógica de negócio" inteira mora nas instructions e na decisão turno a turno do modelo.

Se você sentiu que essa seção não trouxe nada tecnicamente novo, é porque não trouxe de propósito — o objetivo era ver as seções anteriores se somando num cenário fiel ao trabalho real da PCDF, não aprender mais um mecanismo do SDK.

Desafio • CNPJ resiliente dentro do loop encadeado

 DESAFIO BLOQUEADO

A consulta ao CNPJ do 7.1/7.2 pode devolver 429 (rate limit) — a BrasilAPI é gratuita e tem limite de requisições. Reescreva `consultar_cnpj` como `consultar_cnpj_resiliente`, tratando especificamente o status 429 com retry e backoff exponencial (mesmo padrão da 4.1) — sem misturar isso com outros erros HTTP, que devem continuar propagando direto, sem retry. Rode de novo o cenário completo do 7.2 (CNPJ + endereço + registro) e confirme que, mesmo com rate limit temporário, o agente consegue reunir os dois dados antes de pedir aprovação pra registrar a ocorrência.

A solução está protegida por senha — revele no artifact interativo (link no topo da aula).